

WARNING

Be aware that the chips used in your kit are sensitive to static electricity discharge and could be damaged by mishandling of the PCB. Be careful with the board and avoid touching any of the tracks or pads while assembling it. Try to handle it only by the edges.

VERY IMPORTANT

Please do not insert any chip into the programming socket unless the board is connected to the PC either via the serial port or the USB port and the MicroBrn software is running

The controller PICmicro® on the board is held in a reset state until MicroBrn gains control of it. In this reset condition, there will be stray voltages present in the programming socket.

There is a modification that keeps these voltages turned off unless needed by the programmer. Please see “resmod.pdf” in your installation directory. Later version kits have this modification already installed. Even with this modification, it is not advisable to turn the programmer board off and on while a chip is in the programming socket.

Clk & Dat Programming Pins On the programmer board, the PICmicro® driver chip has the Clk and Dat pins set as inputs when the programmer is idle. These are set as outputs only when programming takes place.

Programming a Chip

Programming a chip is straightforward. Follow these steps: 1. Connect the programmer to the PC serial/USB port and turn it on. 2. Run the `MicroBrn.exe` program. Refer to the **Serial Port** section for more details. 3. Select the chip type from the **Chip Selector** (e.g., 16F873). 4. Load a HEX file and ensure the **fuse options** are set correctly. 5. Place the 16F873 chip in the programming socket, paying attention to pin #1's location as shown on the screen. 6. Press **Program**. The programmer will verify the chip type and commence programming if it matches. - If the chip is not blank, you'll be asked if it's okay to overwrite the old data. Choose **Yes** or **No**. - LEDs will light up on the programmer, and progress will be displayed on-screen. 7. You can stop programming anytime by pressing **Cancel**. 8. Once programming is complete, remove the chip—it's ready for use in your project.

Reading a Chip

To read a chip: 1. Place the chip in the programming socket. 2. Select its type from the **Chip Selector**. 3. Press **Read**. The chip type will be verified. 4. If the chip matches, its data will be read and displayed on-screen. You can save this data to disk if needed. -

Note: If the chip is code-protected, the data displayed will be all zeros (0000).

Verifying a Chip

To verify a chip: 1. Place the chip in the programming socket. 2. Select its type from the **Chip Selector**. 3. Press **Verify**. The chip type will be checked, and its data verified. - Any verification errors will trigger an error dialog box.

Fuse Settings

Fuses are used to configure options like oscillator type, watchdog timer enable, power-up timers, etc. Fuse settings vary by chip, and detailed instructions can be found in the respective datasheets.

Example: 16F84

- **Options:** WDT, PWRTE, Oscillator, and Code Protect.
- If your project doesn't need WDT, select **Off** or **Disabled**. For projects requiring WDT, choose **On** or **Enabled**.
- Apply the same method for PWRTE and Code Protect settings.
- Oscillator options include configurations like HS for 4MHz clock speed.
- Programming will use the fuse data to program the fuse register inside the PICmicro® chip.
- **Warning:** Code-protected chips may not be reprogrammable.

Additional Options

- **Merge:** Combine HEX files by adding data to blank addresses only.
- **Refresh:** Reload all HEX files currently in memory.
- **Verify:** Compare HEX code in the data list with the chip.
- **Erase:** Test if the chip is blank or erase FLASH chips.
- **Save:** Save data, including fuses and EEPROM content.
- **Read:** Read data from the selected chip.
- **Load:** Load a HEX file with embedded EEPROM or fuse data.
- **Cancel:** Stop any active programming function.
- **OSCAL:** Manage oscillator calibration values for chips with internal oscillators.
- **USB Port:** Ensure proper connection and shutdown sequence to avoid slowing down your system.
- **Code Protect Warning:** Be cautious when using code protection, as it may make chips non-reprogrammable.
- **Edit HEX Code:** Modify ROM or EEPROM data directly in the list window.

It seems this text also includes corrupted or extraneous characters that obscure its content. Let me reformat and organize the key sections to make the information clearer:

Edit Chip List

This option allows you to modify the chips listed in the **Chip Selector**. - You can reduce the list to only the chips you frequently use, which makes it easier to search. - Chips marked with a 'Y' next to their name will be displayed in the list. - Double-click on 'Y' or 'N' to toggle the visibility.

Fuse Value

This option shows the HEX values of the fuses in the selected chip. - Fuses configure specific chip settings such as the oscillator type, watchdog timer, and power-up features.

Calibration Word(s)

This option displays the calibration word(s) for the chip. - **Note:** You must read a chip first to retrieve its calibration data. - Some chips do not have meaningful calibration data in their memory. - Calibration values are retrieved from addresses `0x2008` and `0x2009`.

For example, the **PIC16F688** has this feature available.

Read Chip Info

This item lets you read the chip's **ID** and **version**. - The chip ID is used by the software to verify the device type before programming. - Some chips, like the **16F84 series**, do not have chip IDs.

Programmer - RESET

Resets the main PICmicro® chip in the programmer by pulling the **MCLR pin** low for 200ms. - If the reset succeeds, you'll see "Board connected." - **Polarity:** Some programmer boards require opposite voltage levels for reset. - If programming or reading fails, ensure the **Programmer Style** setting is correct.

Fly Window

Minimizes the main window and opens a small **fly window**. - This is useful for working alongside tools like **MPLAB®**. - After assembling your code in MPLAB® (e.g., `LEDflash.asm`), the fly window can program the chip with the latest HEX file (`LEDflash.hex`).

Auto Update Files

When enabled, this option automatically checks if the current HEX file has been updated before programming. - If a new version exists, it will be loaded automatically. - If disabled, the file currently on-screen will be programmed.

Recent Files

View and load previously loaded or merged HEX files from this menu.

MPLAB® Integration

Start the MPLAB® IDE from within MicroBrn: - **Start:** Opens MPLAB® if the correct path is set. - **Path:** Configure the location of `MPLAB.exe` (e.g., `C:\Program Files\MPLAB...`).

Clear

Clears the memory buffer. Useful for resetting the workspace.

COM Port

Allows you to change the COM port for serial communication: - Double-click the **COM x** label at the bottom of the screen to open the settings. - Ensure the selected COM port matches the hardware connection. - **Example:** You might select COM1, but your hardware is connected to COM2.

See also: USB Port settings.

Serial Off

Disables the serial port to allow another application to use it while MicroBrn remains running.

ICSP Mode

Enables programming via the **ICSP port**. - **Important Notes:** - The target circuit must not draw significant current from the **VPP pin**. - Only **VPP1** is used in ICSP mode, regardless of chip requirements. - Avoid programming chips in the socket while in ICSP mode to prevent damage.

It seems this text contains some corrupted and extraneous characters, making it difficult to process directly. I've restructured and reformatted the clear parts of the content to enhance readability. Here's the cleaned-up and organized version:

ICSP (In-Circuit Serial Programming) Notes

Refer to the construction PDF in your installation directory for more information on ICSP connections. The file name will match your kit type (e.g., k149.pdf).

Potential ICSP Issues: - High capacitive loads on the programming pins. - Long programming leads. - Inability of the programmer to control the **MCLR** pin or the **VCC voltage**.

If these or other issues occur, isolate the programming pins on the target board.

Auto Program-Enable

This feature allows programming chips without interacting with the Windows interface:

- Insert a chip into the socket; the software detects it and begins the program/verify process. - After programming, insert the next chip to continue. Press **CANCEL** to exit Auto Program mode. - Errors encountered during programming will terminate the mode.

Optional Settings: - Enable **Ignore Blank Check** to skip checking if the chip is blank before programming. This saves time but may cause issues with non-FLASH chips.

Programmer Settings

Match the programmer software to your kit version: - For example, if you have a **K149-A** board, select it from the list. - This is usually done automatically when MicroBrn connects to the programmer. - Some boards allow changing the **Reset Polarity**, which alternates between True or False signals during reset.

Command Line Usage

MicroBrn can be started via a command line. The format is:

MicroBrn.exe [Filename] [Chip Type] [Fly Window switch] [Quit switch]



Examples:

1. Start MicroBrn, load test.hex , use a 16F873 chip, and show the Fly Window: `MicroBrn.exe c:\hexfiles\test.hex 16f873 /F /q`
2. Start MicroBrn, load first.hex , use a 16F84 chip, and close after programming: `MicroBrn.exe c:\hexfiles\first.hex 16f84 /F /q`

```
MicroBrn.exe c:\files\first.hex 16f84 /f /Q
```

Optional: Specify the **Port** on startup. If the port is invalid, MicroBrn will not continue:

```
MicroBrn.exe c:\files\first.hex 16f84 /f /Q 1
```



Protocol Number and Upgrading

Protocol versions from **V210803** and above can be accessed via [HELP] → Protocol . - Matching protocol numbers (e.g., MicroBrn: P18A and Board: P18A) are required for proper functionality.

Upgrading Steps: - Load the HEX file for your programmer board

(e.g., diyk149b.hex for K149-B). - Verify the protocol number in the EEPROM section matches the MicroBrn protocol.

Calibration Program Options

Preserve factory-programmed values for the **OSCAL** and **Bandgap** calibrations:

1. **Message Prompt:** Prompts the user if HEX file values differ from the chip. 2. **Insert Original Into File:** Reads calibration values from the chip and adds them to the HEX file before programming. 3. **Use File:** Programs the HEX file's values directly.

Debug Vector

Programs an address usable as a debug vector for certain **PICmicro® F-series chips**.

Refer to the chip's datasheets for details.

EEPROM Override

This option allows overriding the EEPROM data format in HEX files: - Normally, EEPROM data uses 8 bytes per line for 14-bit cores (e.g., 16F87x family). - Some formats use 16 bytes per line or add 0x00 bytes between data.

Warning: Use with caution, as it can ignore every second byte for 18Fxxx EEPROM data, potentially causing errors.

Problem Support

If your programmer isn't responding, try the following: - Check the **Reset**

Polarity option. Test the unchecked option and try resetting again.

Components and Connections

- **Do not touch the board connections:** Some components may be damaged or degraded by static electricity discharges.
- Verify the construction work:
- Check parts orientation and placement.
- Look for shorted tracks and solder bridges.
- **Kit 149-A:** Ensure diode D3 is correctly placed. Its anode side has a via near the component hole.
- Ensure all polarized components (e.g., electrolytic capacitors and chips) are inserted correctly.
- **Crystal Requirements:** All kits require 6MHz crystals.

- Check the FTDI chip connections to ensure they have been presoldered properly at the factory.

Power Supply

- **Kit 149-A, K149-B, and K150:** Require at least 16VDC, preferably 18VDC, with a supply capable of providing at least 200mA.
- The power connector must have a center-positive connection.
- **Kit 128:** Powered via USB port.
- After powering up, verify the red **PWR LED** is lit. For K128, it will flash red/green before settling to red.

Voltage Measurements:

- **K149 Kits:** Check for 5VDC on test pads using a voltmeter.
- **K150 Kits:** Measure voltage on PICmicro® pins 14 (+ve) and 5 (-ve)—should read 5VDC.
- **Troubleshooting:** If voltage readings do not match, inspect the PCB for issues.

Serial Connection

- **Serial Cable:** Use a straight-through type, not NULL modem. For example:
- Pin 1 to Pin 1, Pin 2 to Pin 2, etc.
- Ensure the COM port is properly selected, open, and connected.
- **Using USB:**
- Verify cable connections.
- Install FTDI drivers (download at [FTDI Driver Site] (<http://www.ftdichip.com/FTWinDriver.htm>)).
- Avoid new enhanced drivers.

Programmer Verification

- Connect the programmer to the PC and apply power.
- Run MicroBrn; you should see **Board Connected** appear in the interface.
- If the programmer works but only programs 1 or 2 locations before reporting connection errors, check if the firmware matches the MicroBrn protocol number (available under **Help** → **About**).

Reading and Error Messages

- If all 0000s appear when reading:
- It may indicate the **Code Protect Fuse** is enabled.
- Ensure the chip is properly inserted into the socket.
- Inspect the chip and PCB for damage.

Troubleshooting Common Issues

1. **K128 Limitations:** Not designed for ICSP programming due to high-impedance voltage supply, but it may work in some cases.
2. **HEX File Requirements:**
 - 12- and 14-bit PICmicro® chips: **INHX8M** or **INHX32** file types.

- 18F family PICmicro® chips: **INHX32** file type only.

3. Flash vs. UV Erase Chips:

- "F" series chips can be programmed normally.
- "C" series chips require JW types and UV erasers.

Low ICSP Programmer Connection

This open collector output may: - Pull **LVP** or **OSC1** pins low while programming. - Serve additional functions on target boards.

Chip ID

- Some PICmicro® chips (e.g., 16F84) lack a chip ID, which may cause error messages during programming.

Additional Notes

- Ensure there are no stray voltages in the programming socket when idle.
- Chips should not get warm in the programming socket—check hardware and insertion.

For PCB modifications related to Kits 149A, 149B, and 150, refer to the "resmod.pdf" file in the installation directory.

ChipInfo File

- This file contains data required by **MicroBrn** to determine chip parameters for programming.
- **MicroBrn V150607** introduced updated `chipdata.txt` to include support for newer PICs, fix errors, and differentiate versions.
- Details of the new `chipdata.txt` are not available.

COPYRIGHT Notice

- © 2006, 2007 **DIY ELECTRONICS** and **Jim Robertson**.
- The MicroBrn software (`MicroPro.exe` , `MicroBrn.exe`), firmware, documentation, and PCB artwork are copyrighted.
- Licensed only for genuine **DIY Electronics PIC programmers** (K128, K182, K149, K150) for programming, reading, verifying, and blank-checking PIC micros.
- **Restrictions:**
- Decompiling, reverse-engineering, or commercial use without permission is prohibited.
- Use of DIY software with non-DIY programmers violates copyright laws.

LIMITATIONS

- DIY programmers are **hobby-level**, not for professional use.
- Newly added PICs in the `chipdata.txt` may contain errors or incomplete support.
- Due to the loss of the original developer, the executable file cannot be updated to correct known bugs.

Known Issues:

- MicroBrn may write code memory to the EEPROM on **18Fxxx** series chips.

- Config word, ID values, and EEPROM data may not save/display correctly.
- Cannot program chips with more than 64k bytes of data.
- Timing for code and data EEPROM programming is not independent.

Firmware Updates and Compatibility

- The firmware's protocol identifier is **P18A**.
- New firmware and `chipdata.txt` must be used together and not mixed with previous versions.
- Renamed files (`chipdata.txt` from `chipinfo.dat`) ensure compatibility.

Known (Unfixable) Problems in MicroBrn (Version v281004)

- Supports up to 34 configuration settings, which may not be enough for newer chips.
- Issues with saving/displaying specific data (config words, EEPROM, etc.).
- **18Fxxxx Family**: Can cause syntax errors for parts exceeding 64k bytes.

PIC10F Series Support

- "Beta support" for PIC10F was introduced in version v160804.
- Support was later abandoned, and these parts are no longer included in the `chipdata.txt`.

ERASE Function

- **Erase Check**: Verify the chip is erased.
- **Erase Chip**: Erase FLASH-type chips.

Additional Notes

- Report any errors in the chipdata file to the DIY forum: [DIY Forum] (<http://www.websitetoolbox.com/tool/mb/diykit?forum=13943>).